

再帰下降パーサ② — 文を解析する

今日のゴール

F2 の式パーサを継承して、文 (`return` / `break` / `continue` / `if` / `while` / `for` / ブロック / 式文) の解析を追加する。37 本の文コーパスで、AST が scaffold の Parser と完全一致することを golden test で確認する。

1 この回の位置づけ

- フロントエンド発展シリーズの第 3 回 (選択制)。前提は F2
- StmtParser は、自分が F2 で作った ExprParser を importlib で継承する。本編の CodegenNN と同じ「前回の自分を継承する」方式である。F2 が未完成だとこの回のテストは動かない
- 宣言 (`int x;` など) と関数定義は F4 で扱う。この回のブロックは「文だけを含むブロック」である

2 文は先頭のトークンで決まる

式は 10 段の優先順位の階段が必要だった。文はずっと素直で、先頭のトークンを見るだけで種類が確定する。

先頭のトークン	文の種類	呼ぶ関数
<code>return</code>	return 文	<code>parse_return</code>
<code>break</code> / <code>continue</code>	ジャンプ文	<code>parse_break</code> / <code>parse_continue</code>
<code>if</code>	条件分岐	<code>parse_if</code>
<code>while</code> / <code>for</code>	ループ	<code>parse_while</code> / <code>parse_for</code>
<code>{</code>	ブロック	<code>parse_block</code>
それ以外	式文 (空文 ; 含む)	<code>parse_exprstmt</code>

スケルトンの `parse_stmt` は、この表のと通りのディスパッチとして完成済みである。迷う要素がないので、先読みは現在のトークン 1 つで足りる (LL(1))。

3 単純な文 — ; を食べ忘れない

`return / break / continue` / 式文は、EBNF がそのままコードになる。

```
'return' [ expr ] ';' ;
```

```
def parse_return(self):
    self.pos += 1          # 'return'
    if self.consume_if(';'):
        return Node(ND_RETURN)      # return;
    expr = self.parse_expr()
    self.expect(';')
    return Node(ND_RETURN, operand=expr)  # return expr;
```

ここで `parse_expr` は F2 で自分が書いた関数である。文のパーサは、式の中身を一切知らずに「式を 1 個読んでくれ」と依頼するだけでよい。文と式で層が分かっているおかげである。

ありがちなバグは ; の食べ忘れである。`expect(';')` を忘れると、その ; が次の文の先頭に残り、1 つずれた場所で意味不明なエラーになる。

4 ブロック — 文の再帰の輪

```
block ::= '{' { stmt } '}'
```

ブロックは文の並びであり、ブロック自身も文である。つまり `parse_block` → `parse_stmt` → `parse_block` の再帰の輪ができる。式のときの (`expr`) とまったく同じ構図で、これが { { `return 1;` } } のような入れ子を自然に処理する。

} が来るまで `parse_stmt` を繰り返して `stmts` リストに集め、`Node(ND_BLOCK, stmts=...)` を作る。入力が尽きても } が来ない場合のエラーも忘れずに。

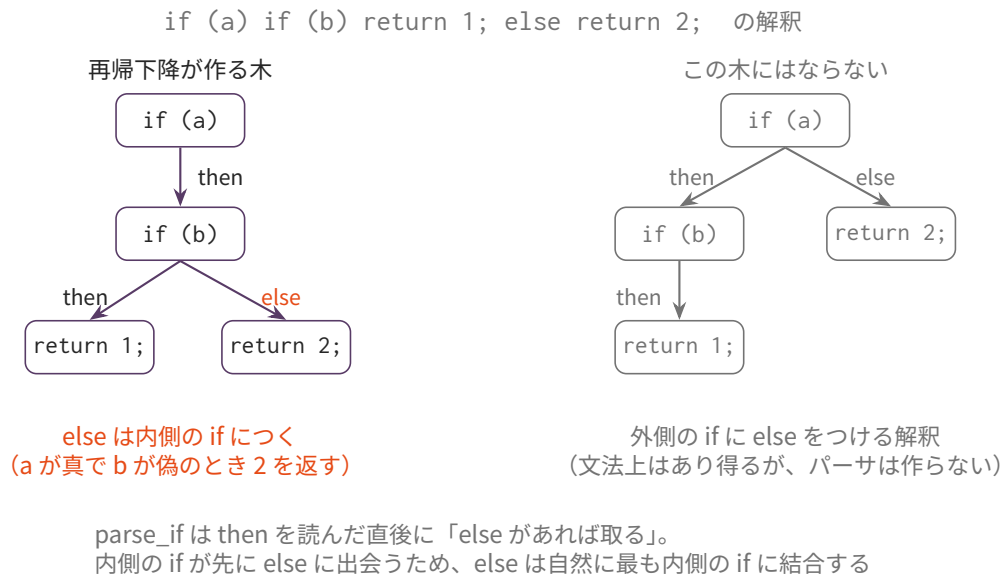


図 1 dangling else — else は最も内側の if に結合する

5 if と dangling else

```
if_stmt ::= 'if' '(' expr ')' stmt [ 'else' stmt ]
```

then 側は `parse_stmt` で読む。ブロックとは限らず、単文でも入れ子の `if` でもよい。そして then を読んだ直後に、**else があれば取る**。実装はこれだけである。

```

then = self.parse_stmt()
else_ = None
if self.consume_if('else'):
    else_ = self.parse_stmt()
        
```

ここで、コマ 5 の資料にあった「**else は最も内側の if に結合する**（最近傍優先）」という規則の種明かしができる。

`if (a) if (b) return 1; else return 2;` を読むとき、**else** に最初に出会うのは内側の `if` を読んでいる最中の `parse_if` である。「else があれば貪欲に取る」だけで、最近傍優先が特別な処理なしに実現される。曖昧になりうる文法が、実装の素直な形によって一意に決まっている例である。

また、`else if` という専用の構文がないことも実装から分かる。`else` の後の `stmt` がたまたま `if` 文である、というだけである（コマ 5 「else if の扱い」の実装側）。

6 while と for — 省略可能要素は None

`while` は `if` から `else` を除いた形で、新しい要素はない。

`for` は 3 つの要素がそれぞれ省略できる。

```
for_stmt ::= 'for' '(' [expr] ';' [expr] ';' [expr] ')' stmt
```

省略の判定は「次の区切り記号が来ていたら中身なし」で行う。

要素	省略の判定	省略時
init	次が <code>;</code> か	None
cond	次が <code>;</code> か	None
step	次が <code>)</code> か	None

コマ 6 の資料に「省略された部分は AST 上で `None` になる」とあった。その `None` を作っているのがこのコードである。`for (;;)` は 3 要素とも `None` の `ND_FOR` になり、コード生成側 (コマ 6) が「`cond` が `None` なら `beqz` を出さない」と対応していた。

7 Node の形

S 式では、文は次のように表示される (`parse_viewer` と同じ)。

```
(if (cond (var "x")) (then (return (num 1))))
(for (init none) (cond none) (step none) (body (break))))
```

この回で作るノードとフィールドの対応をまとめておく。

文	kind	使うフィールド
<code>return [e];</code>	<code>ND_RETURN</code>	<code>operand</code> (なければ <code>None</code>)
<code>break;</code> / <code>continue;</code>	<code>ND_BREAK</code> / <code>ND_CONTINUE</code>	—
<code>e; / ;</code>	<code>ND_EXPRSTMT</code>	<code>operand</code> (空文は <code>None</code>)

文	kind	使うフィールド
<code>{ ... }</code>	<code>ND_BLOCK</code>	<code>stmts</code> (リスト)
<code>if</code>	<code>ND_IF</code>	<code>cond</code> , <code>then</code> , <code>else_</code>
<code>while</code>	<code>ND_WHILE</code>	<code>cond</code> , <code>body</code>
<code>for</code>	<code>ND_FOR</code>	<code>init</code> , <code>cond</code> , <code>step</code> , <code>body</code>

8 実装

Step	実装対象	内容
1	<code>parse_return</code> / <code>parse_break</code> / <code>parse_continue</code> / <code>parse_exprstmt</code>	単純な文。 ; を食べ忘れない
2	<code>parse_block</code>	文の再帰の輪
3	<code>parse_if</code>	dangling else
4	<code>parse_while</code> / <code>parse_for</code>	省略可能要素はNone

ディスパッチ (`parse_stmt`) と入口 (`parse_statement`) は完成済み。途中経過は目視でも確認できる。

```
python3 myparser.py 'if (a > b) return a; else return b;'
```

9 テスト

```
python3 check.py      # Step ごとの単体テスト (期待値は S 式)
python3 golden.py     # 37 本の文コーパスで scaffold と突き合わせ
```

`golden` のコーパスには、講義のテストに出てきた形 (`while` の総和、連結リストの走査、`continue` でスキップする `for` など) を含めてある。全文 **PASS** がこの回の完了条件である。

10 発展課題

1. **do-while** を追加する: `do stmt while '(' expr ')' ';' ;` (Core 外。golden は対象外)。条件を後で読むのに AST はどう持つべきか考える
2. 意地悪な入力: `if (a) else return 1;` (then がない) や `while () x;` (条件がない) にどんなエラーを出すか確かめ、メッセージを改善する
3. 文の数を数える: `parse_stmt` が呼ばれた回数を数え、プログラムの文の個数と一致することを確かめる

補足

コラム: 文の解析はなぜ簡単なのか。C の文法は、文の先頭に必ず「区別できる目印」(キーワードか{) が来るように設計されている。この性質のおかげで、文のパースはバックトラックなしの単純なディスパッチで書ける。一方、宣言 `int x;` と式文 `x;` の区別は「先頭が型名かどうか」に依存し、`typedef` で型名が増えるため、トークンの種類だけでは決められない。これが F4 で扱う、C の文法の一番厄介な部分である。

11 次回予告

残るは宣言と型である。F4 では `parse_type` ・ローカル/グローバル宣言・配列サフィックス・関数定義/宣言・`typedef` ・`sizeof` を実装し、`parse_program` を完成させる。そして全テスト入力(約 100 本の .c)で scaffold と AST を突き合わせ、黒箱の完全な置き換えを宣言する。